

Autonomous Robots Report



“Autonomous Exploration and Pillar-to-Pillar Navigation in Webots Environments”

Submitted by

Aesha Gadhiya
Rinkal Zadafiya
Umang Dholakiya

Supervised by

Prof. Dr.-Ing. Thomas Nierhoff

Ostbayerische Technische Hochschule Amberg-Weiden

Department of Electrical Engineering,
Media and Computer Science

July 10, 2026

1 Approach

1.1 Overview

This project implements a complete online navigation controller for a four-wheeled RosBot in the Webots simulator. The robot is placed in an initially unknown maze and must reach two coloured landmarks in a fixed order: first the **blue** pillar, then the **yellow** pillar. It must avoid ordinary and floating walls, must never drive onto the **green** floor (a forbidden “poison” region), and must operate without the Webots Supervisor and without modifying the robot or the world. Because the map is unknown at start-up, no route can be precomputed; the controller therefore builds its own map while moving and plans on that map as it grows.

The controller works in two coupled phases. During *exploration* it maps the maze and searches for the pillars by driving to the boundary between known and unknown space. Once a required pillar is both localised and connected to the robot through known free space, the controller switches to *goal-directed navigation*, plans a route with A*, and follows it. The same map, planner and motion controller are reused across both phases, so the robot never keeps two inconsistent representations of the world.

1.2 Strategy overview

The design rests on four principles.

1. Synchronous perception every control step. Unlike a multi-threaded design, the whole pipeline — odometry, LiDAR mapping, camera segmentation, planning and motion — runs sequentially inside a single Webots control loop. Every subsystem therefore reads the same map state at the same simulation time, which removes race conditions and makes the behaviour deterministic and repeatable.

2. Frontier-based exploration limited to reachable space. Rather than wandering randomly, the robot always targets *frontiers* — free cells that touch unknown space. Crucially, a frontier is only chosen if it is reachable from the robot through known free space, measured by breadth-first search in *path* distance rather than straight-line distance. This prevents the classic failure of steering toward a visually near frontier that is actually behind a wall.

3. A mission state machine that enforces the required order. A small state machine guarantees the blue-then-yellow sequence. Yellow observations may be collected early to save time later, but they can never trigger yellow navigation before the blue pillar has been reached.

4. Map-coupled planning and replanning. Perception and motion are coupled only through the occupancy map. A pillar estimate is accepted only when a free navigation goal can be formed near it; a path is invalidated and recomputed whenever new mapping evidence (a new wall, or a newly marked green region) blocks the active route.

1.3 Software structure

The controller is implemented as a single class, `RosbotExplorer`, in `my_controller.py`. All state — the log-odds grid, the discrete planning map, odometry, mission flags and pillar estimates — is held in one object, and the top-level `while` loop calls the subsystem methods in a fixed order on every step. Table 1 lists the main responsibilities.

The robot devices used are four skid-steer wheel motors with their position sensors (used as encoders), an inertial unit for yaw, a 2-D LiDAR, an RGB camera and an aligned depth camera, plus a map display for diagnostics. The simulation control period is the world basic time step

Table 1: Main software responsibilities inside `RosbotExplorer`.

Group of methods	Responsibility
Device setup / stepping	Motor, encoder, IMU, LiDAR, RGB/depth camera and display initialisation; one synchronous simulation step.
Pose estimation	Supervisor-free odometry from wheel encoders and inertial yaw.
Mapping	Log-odds occupancy update with Bresenham ray tracing and temporal filtering.
Perception	HSV colour segmentation, pillar bearing/range fusion, green-floor handling.
Exploration	Frontier detection and reachability search.
Planning	Hard/soft inflation, A* search, line-of-sight waypoint tracking.
Motion & safety	Proportional waypoint controller and reactive front-sector protection.
Mission logic	State machine enforcing blue-first ordering, replanning and stopping.

(32 ms). No Supervisor node and no ground-truth pose are used at any point.

2 Pose estimation and coordinate frames

Because the Supervisor is forbidden, the pose is dead-reckoned. Translation comes from the four wheel encoders and heading is taken from the inertial unit, with differential-drive odometry as a fallback when yaw is unavailable. Writing the wheel-radius-scaled increments of the left and right sides as d_L and d_R , the forward step and pose update are

$$d = \frac{1}{2}(d_L + d_R), \quad \theta_k = \theta_k^{\text{IMU}} \quad (\text{else } \theta_{k-1} + \frac{d_R - d_L}{b}), \quad (1)$$

$$x_k = x_{k-1} + d \cos \theta_k, \quad y_k = y_{k-1} + d \sin \theta_k, \quad (2)$$

with wheel radius $r_w = 0.043$ m and odometric axle track $b = 0.26$ m. Tiny encoder increments during near-pure rotation are suppressed so that spinning in place does not create spurious translation. The pose is expressed in a robot-local odometry frame that starts at the origin; all LiDAR endpoints, camera projections and planning goals live in this one frame, so absolute world coordinates are never needed. A world point (x, y) is mapped to a grid index by

$$m_x = \frac{M}{2} + \left\lfloor \frac{x}{\rho} \right\rfloor, \quad m_y = \frac{M}{2} - \left\lfloor \frac{y}{\rho} \right\rfloor, \quad (3)$$

with $M = 400$ cells and cell size $\rho = 0.02$ m. The map is only *locally* metric: there is no loop closure, so heading and slip errors can slowly displace older structures. The inertial yaw and the small, bounded maze sizes keep this drift acceptable in practice.

3 Probabilistic occupancy mapping

3.1 Grid and log-odds update

The environment is represented by an $8 \text{ m} \times 8 \text{ m}$ grid at 0.02 m resolution, i.e. 400×400 cells. Each cell stores a log-odds value, and a separate discrete state records whether it is *unknown*, *free* or *occupied* for planning. Following the standard occupancy-grid model, each LiDAR beam

decreases the log-odds of the cells it passes through and increases the log-odds of the cell it ends on:

$$L(m) \leftarrow \text{clip}(L(m) + \Delta L, -6, +6), \quad p(m) = \frac{1}{1 + e^{-L(m)}}, \quad (4)$$

with a free update of $L_{\text{free}} = \log(\frac{0.35}{0.65})$ along the ray and an occupied update of $L_{\text{occ}} = \log(\frac{0.9}{0.1})$ at the endpoint. The log-odds are clipped to $[-6, 6]$ to keep the map able to “forget” and to prevent unbounded growth.

3.2 From evidence to a planning state

The continuous probability is converted to the discrete state through confirmation counters. A cell becomes *free* after a single free observation, but only becomes *occupied* after two consistent occupied observations. This asymmetry is deliberate: free space should open quickly so the planner can use it, whereas occupied cells are held back to suppress the isolated speckles that appear between sparse LiDAR rays. In addition, each cell is updated at most once per scan identifier, which stops overlapping beams within a single scan from over-inflating a wall.

3.3 Ray tracing that protects narrow corridors

Free space is written along each beam with Bresenham’s integer line algorithm. One important detail is that the free ray is terminated *half a cell before* the measured range, and the true endpoint is marked occupied separately. If instead the shortened endpoint were marked occupied, walls would systematically thicken toward the robot and could turn a physically traversable corridor into a blocked one. Mapping is skipped entirely when both the translation and the yaw change since the last update are below small thresholds, which saves computation while still allowing updates during in-place rotation. Figure 1 shows a raw map produced by the controller.

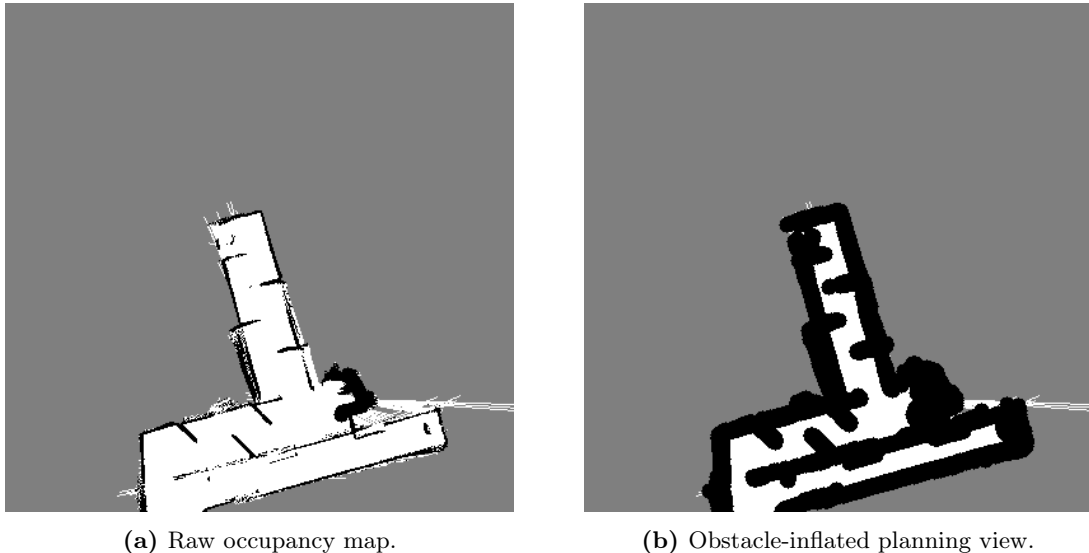


Figure 1: Map artefacts saved by the submitted controller (Maze 5). White is confirmed free space, dark cells are occupied, and grey is still unknown. Unknown, free and occupied regions are separated *before* the clearance inflation of the planner is applied.

4 Semantic perception and environmental modelling

4.1 Colour segmentation

The RGB frame is converted to HSV, since hue separates the target colours more reliably than raw RGB. To reduce edge noise and to keep the floor in view for green detection, processing is restricted to a central region of interest whose vertical centre is shifted slightly downward. The implemented thresholds are given in Table 2; they are fixed configuration values, not a learned model. For each colour, the connected-mask coverage and centroid are computed, and all colours above a small area threshold are returned together so that, for example, a blue pillar is not dropped when green is also visible.

Table 2: HSV segmentation thresholds (hue in degrees, saturation/value in $[0, 1]$).

Class	Hue	Saturation	Value
Blue pillar	200°–250°	≥ 0.4	≥ 0.3
Yellow pillar	40°–70°	≥ 0.4	≥ 0.4
Green floor	80°–140°	≥ 0.4	≥ 0.3
Red wall	$\leq 20^\circ$ or $\geq 340^\circ$	≥ 0.4	≥ 0.3

4.2 Pillar bearing, range fusion and ordering

The horizontal image displacement of a detected centroid gives a bearing relative to the camera. The range is read from a small window of the depth camera around the centroid and, where useful, fused with the LiDAR range at the same bearing by taking the smaller valid value — this protects against the depth camera occasionally reading the background behind a thin pillar. For robot pose (x_r, y_r, θ_r) , bearing α and range d , the pillar estimate is

$$\begin{bmatrix} x_o \\ y_o \end{bmatrix} = \begin{bmatrix} x_r \\ y_r \end{bmatrix} + d \begin{bmatrix} \cos(\theta_r + \alpha) \\ \sin(\theta_r + \alpha) \end{bmatrix}. \quad (5)$$

The actual navigation goal is offset by a small standoff toward the robot, so that A^* targets a free cell in front of the pillar rather than the occupied pillar surface. Blue is committed as soon as a usable estimate exists (above roughly 1% coverage), which allows the robot to react to a pillar seen far away during the initial scan. Yellow is treated more carefully: several consecutive estimates are collected and averaged, and the sample set is rejected if its spatial spread is too large, because a small bearing error can otherwise move the projected goal across a wall boundary and make the route flicker in and out of existence. The blue-first order is enforced purely in the state machine: yellow can be *seen* and stored at any time, but it can only be *navigated to* after blue is reached.

4.3 Green-floor handling

Green floor is not visible to a horizontal LiDAR, so it is converted from a camera observation into occupied map cells. To avoid polluting the map with false green, cells are only marked when green appears low and close in the camera image, and marked cells are forced occupied and kept in a protected set so that later LiDAR free-space rays cannot erase them. Green is deliberately stored using the *same* occupied representation as walls, which means frontier reachability, A^* , waypoint line-of-sight and path invalidation all reject it without needing a separate semantic planner. While driving to blue, a slightly larger “poison” area is blocked around a green detection

to push replanning away from it, and any active path that begins to cross a newly marked green region is immediately cleared and recomputed.

5 Frontier-based exploration

A frontier cell is a free cell that has at least one unknown neighbour. Frontier cells may be grouped into clusters, but the target is not chosen by a blind cluster centroid, because rounding can place a centroid inside an unknown or occupied cell. Instead, the controller runs a four-connected breadth-first search outward from the robot through known free space and selects the first frontier it reaches. This returns the nearest frontier in *traversable path distance*, which correctly ignores frontiers that are close in a straight line but blocked by a wall or lying inside the robot-clearance zone. A minimum separation is applied so the robot does not fixate on frontiers right next to it, and repeatedly unreachable targets are blacklisted so exploration does not stall on them. When a pillar has been seen but is not yet connected, a target-biased variant prefers frontiers that lead toward the pillar, so exploration spends its time opening the part of the map that is most likely to unlock the route. If strict clearance temporarily disconnects every frontier, the search is retried with the hard-inflation restriction relaxed, which keeps the robot moving instead of spinning in place forever.

6 Global path planning with A*

Global routes are computed with A* on the discrete planning map. The search is four-connected with a Manhattan-distance heuristic, and it expands *only* cells already classified as free — it never plans through unknown space. This is a conservative but safe choice: the robot will not take a speculative shortcut through unmapped territory, at the cost of having to explore a full free-space connection before a pillar route becomes available. The target-biased frontier mechanism above is what pays that cost back efficiently.

Robot clearance is imposed as a cost rather than a single hard radius. For each candidate cell, the planner finds the nearest occupied cell within a soft radius r_s and applies

$$c_{\text{soft}} = w_s \cdot \frac{r_s - d_o}{r_s - r_h}, \quad c_{\text{soft}} \in [0, w_s], \quad (6)$$

where d_o is the distance to that obstacle, r_h is a hard radius inside which the cell is forbidden, and w_s weights how strongly routes are pushed toward corridor centres. The submitted values are $r_h = 0.11$ m, $r_s = 0.22$ m and $w_s = 150$ (Maze 2 uses a slightly wider $r_h = 0.13$ m, $r_s = 0.24$ m for its tighter geometry). Hard inflation removes connectivity through gaps that cannot physically contain the robot, while soft inflation keeps narrow-but-valid corridors connected and simply makes wall-hugging routes expensive. This is important because a single large binary inflation would be safe in open space but would wrongly delete the only valid narrow passage. If the inflated search fails, a relaxed pass without inflation is attempted over free cells so the robot can still move through very tight fresh maps; the start and goal cells are always snapped to the nearest free cell first.

7 Path following and local safety

7.1 Line-of-sight waypoint tracking

A raw four-connected A* path zig-zags cell by cell. Instead of tracking every cell or smoothing with a spline that might cut a corner, the controller looks ahead up to a fixed number of path cells and drives to the *farthest* one that is connected to the robot by a collision-free Bresenham line through free cells. This removes redundant intermediate steps and produces smooth motion, but it can never cut through an obstacle merely to look smooth, because every cell on the line-of-sight segment must remain free.

7.2 Proportional control and turning

For the selected waypoint at distance d_g and heading error e_θ , the commands are $v = k_v d_g$ and $\omega = k_\omega e_\theta$ with $k_v = 5.0$ and $k_\omega = 4.0$, saturated at the robot's velocity limits and converted to left/right wheel commands by $v_L = v - \omega$, $v_R = v + \omega$. When the heading error is large, the robot performs a short forward pre-turn and then rotates in place before resuming forward motion; this places it away from a wall endpoint before a sharp rotation and reduces corner clipping in narrow layouts.

7.3 Reactive front-sector protection

Because the map is still being built, a close object can be detected by the current LiDAR scan before enough occupied confirmations have propagated into the planning map. A deterministic safety layer therefore inspects a front sector of about $\pm 20^\circ$: if the nearest valid range drops below 0.12m the robot briefly reverses, and below 0.25m its forward and angular speeds are capped. This layer is an override around the global follower, not a second planner.

8 Mission state machine and replanning

The high-level behaviour is a state machine with the states *initial_scan*, *go_to_blue*, *search_yellow*, *explore*, *go_to_yellow* and *done* (Figure 2). At start-up the robot performs a full in-place scan to expose nearby geometry and to look for pillars; this scan is intentionally *not* cut short when blue appears, so the camera also has a chance to see yellow. Exploration is pre-empted the moment the currently required pillar is localised and a path to it can be planned. In the two navigation states the controller keeps a target-specific path and clears it whenever a mapped obstacle or a newly marked green region intersects it, replanning from the current pose. Repeated planning failures fall back to further exploration rather than giving up. Reaching yellow sets the *done* state and stops the wheels.

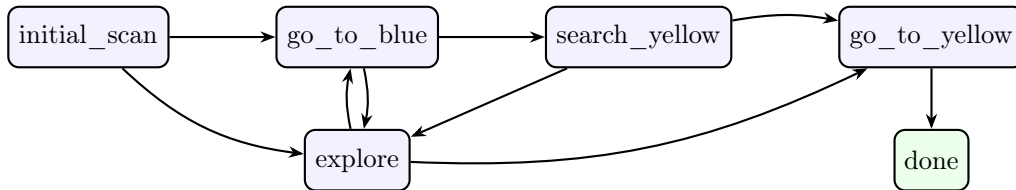


Figure 2: Mission state machine. Blue is always reached before yellow; yellow observations may be accumulated at any time but cannot advance the mission until blue has been reached.

9 Results

Performance is measured in Webots simulation time. The first interval runs from simulation start until the controller reports the blue pillar reached; the second runs from that event until yellow is reached and the controller enters *done*. Table 3 reports the recorded timings for this submission (Maze 2, Maze 3, Maze 4 and Maze 5), read from the simulation clock in the recorded videos.

Table 3: Navigation timing for the recorded runs (mm:ss.mmm; seconds in brackets).

Maze	Start → Blue	Blue → Yellow
Maze 2	3:20.345 (200.345 s)	7:41.231 (461.231 s)
Maze 3	1:50.035 (110.035 s)	4:01.007 (241.007 s)
Maze 4	3:23.001 (203.001 s)	2:32.615 (152.615 s)
Maze 5	3:05.024 (185.024 s)	1:31.296 (91.296 s)

Across the recorded mazes, the longer interval is usually the exploration phase that precedes the blue pillar, which is expected: the planner refuses to cross unknown space, so a complete free-space connection must be mapped before any route can be executed. Once the map is open, the blue-to-yellow leg is short because both pillar estimates are already available and only the final route has to be planned and followed.

10 Limitations

The approach has known boundaries that also define where its parameters must be tuned.

- **Dead-reckoned pose.** The map is locally metric with no loop closure, so encoder slip and heading error can slowly distort older structures over long trajectories. This is mitigated by inertial yaw and the small maze sizes but not eliminated.
- **Green depends on vision.** Green marking relies on camera segmentation and ground projection; strong lighting changes or partial occlusion can delay it. The thresholds are chosen to prefer a false “too close” trigger over missing green entirely.
- **Two-dimensional floating walls.** A floating wall is handled correctly only when it intersects the horizontal LiDAR plane. A surface entirely above or below the scan height would not be reconstructed by the current mapping subsystem.
- **Conservative planning.** Because A* uses only known-free cells, a route can be temporarily unavailable even when a short unknown shortcut is physically open; the cost is extra exploration time.
- **Per-maze tuning and unrecorded runs.** Clearance parameters were tuned per maze (e.g. wider inflation for Maze 2’s tighter corridors). Maze 2, Maze 3, Maze 4 and Maze 5 are reported here with the verified simulation-clock timings available from the recordings; runs for the remaining mazes were not included in this submission and no timing has been fabricated for them.

11 Declaration of AI Usage

AI-based tools were used as supporting resources during the development of the controller and the preparation of this report. ChatGPT and Claude were used for code guidance and assistance

during implementation, while Perplexity was used for analysis and documentation support. These tools assisted with debugging, improving technical explanations, and organizing the report.

References

- [1] B. Yamauchi, “A Frontier-Based Approach for Autonomous Exploration,” in *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation*, pp. 146–151, 1997. Available: https://www.cs.cmu.edu/~motionplanning/papers/sbp_papers/integrated1/yamauchi_frontiers.pdf
- [2] A. Elfes, “Using Occupancy Grids for Mobile Robot Perception and Navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989. Available: https://www.cs.cmu.edu/~motionplanning/papers/sbp_papers/integrated4/elfes_occup_grids.pdf
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. Available: <https://ai.stanford.edu/~nilsson/OnlinePubs-Nils/PublishedPapers/astar.pdf>
- [4] noshluk2, *ROS Navigation Stack and SLAM for Autonomous Custom Robot*. GitHub repository. Available: <https://github.com/noshluk2/ROS-Navigation-Stack-and-SLAM-for-Autonomous-Custom-Robot>
- [5] Husarion, *rostop-autonomy*. GitHub repository. Available: <https://github.com/husarion/rostop-autonomy>
- [6] Husarion, *rostop*. GitHub repository. Available: <https://github.com/husarion/rostop>
- [7] Husarion, *rostop_ros*. GitHub repository. Available: https://github.com/husarion/rostop_ros